

P3505R0

Fix the default floating-point representation in `std::format`

Published Proposal, 2025-03-16

Authors:

[Victor Zverovich](#)

[Junekey Jeon](#)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Table of Contents

1 Introduction

2 Problem

3 Proposal

4 Wording

5 Implementation and usage experience

6 Impact on existing code

References

Informative References

"Is floating-point math broken?" - Cato Johnston ([Stack Overflow](#))

§ 1. Introduction

When `std::format` was proposed for standardization, floating-point formatting was defined in terms of `std::to_chars` to simplify specification. While being a positive change overall, this introduced a small but undesirable change compared to the design and reference implementation in [\[FMT\]](#), resulting in surprising behavior to users, performance regression and an inconsistency with other mainstream programming languages that have similar facilities. This paper proposes fixing this issue, bringing the floating-point formatting on par with other languages and in line with the original design intent.

§ 2. Problem

Since Steele and White's seminal paper ([\[STEELE-WHITE\]](#)), based on their work in the 70s, many programming languages converged on similar default representation of floating point numbers. The properties of an algorithm that produces such a

representation are formulated in the paper as follows:

- No information is lost; the original fraction can be recovered from the output by rounding.
- No "garbage digits" are produced.
- The output is correctly rounded.
- It is never necessary to propagate carries on rounding.

The second bullet point means that the algorithm shouldn't produce more decimal digits (in the significand) than necessary to satisfy the other requirements, most importantly the round-trip guarantee. For example, `0.1` should be formatted as `0.1` and not `0.10000000000000001` even though they produce the same value when read back into an IEEE 754 `double`.

The last bullet point is more of an optimization for retro computers and is less relevant on modern systems.

[STEELE-WHITE] and papers that followed referred to the second criteria as "shortness" even though it only talks about the number of decimal digits in the significand and ignores other properties such as the exponent and the decimal point.

Once such shortest decimal significand and the corresponding exponent are known the formatting algorithm normally chooses between fixed and exponential representation based on the value range. For example, in Python ([PYTHON-FORMAT]) and Rust (the `?f` format which gives the "shortest" representation for FP numbers) if the decimal exponent is greater or equal to 16 for `double`, the number is printed in the exponential format:

```
>>> 1234567890123456.0
1234567890123456.0
>>> 12345678901234567.0
1.2345678901234568e+16
```

16 is a reasonable threshold because IEEE 754 double precision can represent most 16-digit decimal values with high fidelity, and it balances human readability with precision retention when switching between fixed and exponential notation.

[FMT], which is modeled after Python's formatting facility, adopted a similar representation based on the exponent range.

Swift has similar logic, switching to exponential notation for numbers greater or equal to 2^{53} (9007199254740992). This is also a reasonable choice although a threshold that is not a power of 10 might be less intuitive to some users.

Similarly, languages normally switch from fixed to exponential notation when the absolute value is smaller than some small decimal power of 10, usually 10^{-3} ([JAVA-DOUBLE]) or 10^{-4} (Python, Rust, Swift).

When `std::format` was proposed for standardization, floating-point formatting was defined in terms of `std::to_chars` to simplify specification with the assumption that the latter follows the industry practice for the default format described above. It was great for explicit format specifiers such as `e` but, as it turned out recently, it introduced an undesirable change to the default format. This problem is that `std::to_chars` defines "shortness" in terms of the number of characters in the output which is different from the "shortness" of decimal significand normally used both in the literature and in the industry.

The exponent range is much easier to reason about. For example, in this model `100000.0` and `120000.0` are printed in the same format:

```
>>> 100000.0
100000.0
>>> 120000.0
120000.0
```

However, if we consider the output size the two similar numbers are now printed completely differently:

```
auto s1 = std::format("{} ", 100000.0); // s1 == "1e+05"  
auto s2 = std::format("{} ", 120000.0); // s2 == "120000"
```

It seems surprising and undesirable.

Note that the output `1e+05` is not really of the shortest possible number of characters, because `+` and the leading zero in the exponent are redundant. In fact, those are required, according to the specification of `to_chars` ([\[charconv.to.chars\]](#)),

value is converted to a string in the style of `printf` in the "C" locale.

and the exponential format is defined as follows by the C standard ([\[N3220\]](#)):

A double argument representing a floating-point number is converted in the style `[-]d.ddd e±dd ...`

Nevertheless, users interpreting the shortness condition too literally may find this surprising.

Even more importantly, the current representation violates the original shortness requirement from [\[STEELE-WHITE\]](#):

```
auto s = std::format("{} ", 1234567890123456700000.0);  
// s == "1234567890123456774144"
```

The last 5 digits, `74144`, are what Steele and White referred to as "garbage digits" that almost no modern formatting facilities produce by default. For example, Python avoids it by switching to the exponential format as one would expect:

```
>>> 12345678901234567800000.0  
1.2345678901234568e+22
```

The current behavior is a consequence of the shortness-in-characters criterion favoring the fixed format for large numbers while still satisfying the correct rounding condition ([\[charconv.to.chars\]](#)):

If there are several such representations, the representation with the smallest difference from the floating-point argument value is chosen, resolving any remaining ties using rounding according to `round_to_nearest` ([\[round.style\]](#)).

Apart from giving a false sense of accuracy to users it also has negative performance implications. Many of the optimized float-to-string algorithms based on Steele and White's criteria, such as Dragonbox ([\[DRAGONBOX\]](#)) and Ryu ([\[RYU\]](#)), only focus on those criteria, especially the shortness of decimal significand rather than the number of characters. As a result, an implementation of the default floating-point handling of `std::format` (and `std::to_chars`) cannot just directly rely on these otherwise perfectly appropriate algorithms. Instead, it has to introduce non-trivial logic dedicated for computing these "garbage digits". Furthermore, having to introduce dedicated logic is likely not just because of the lack of advancement in the algorithm research, because in this case we do need to compute more digits than the actual precision implied by the data type, thus it is natural to expect that we may need more precision than the case without garbage digits. (In other words, even though a new algorithm that correctly deals with this garbage digits case according to the current C++ standard is invented, it is likely that it still includes some special handling of that case, in one form or another.)

The performance issue can be illustrated on the following simple benchmark:

```
#include <format>  
#include <benchmark/benchmark.h>  
  
// Output: "1.2345678901234568e+22"  
double normal_input = 1234567890123456700000.0;
```

```

// Output (current): "1234567890123456774144"
// Output (desired): "1.2345678901234568e+21"
double garbage_input = 1234567890123456700000.0;

void normal(benchmark::State& state) {
    for (auto s : state) {
        auto result = std::format("{", normal_input);
        benchmark::DoNotOptimize(result);
    }
}

BENCHMARK(normal);

void garbage(benchmark::State& state) {
    for (auto s : state) {
        auto result = std::format("{", garbage_input);
        benchmark::DoNotOptimize(result);
    }
}

BENCHMARK(garbage);

BENCHMARK_MAIN();

```

Results on macOS with Apple clang version 16.0.0 (clang-1600.0.26.6) and libc++:

```

% ./double-benchmark
Unable to determine clock rate from sysctl: hw.cpusfrequency: No such file or directory
This does not affect benchmark measurements, only the metadata output.
***WARNING*** Failed to set thread affinity. Estimated CPU frequency may be incorrect.
2025-02-02T08:06:13-08:00
Running ./double-benchmark
Run on (8 X 24 MHz CPU s)
CPU Caches:
  L1 Data 64 KiB
  L1 Instruction 128 KiB
  L2 Unified 4096 KiB (x8)
Load Average: 7.61, 5.78, 5.16

```

Benchmark	Time	CPU	Iterations
normal	77.5 ns	77.5 ns	9040424
garbage	91.4 ns	91.4 ns	7675186

Results on GNU/Linux with gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0 and libstdc++:

```

$ ./double-benchmark
2025-02-02T17:22:25+00:00
Running ./double-benchmark
Run on (2 X 48 MHz CPU s)
CPU Caches:
  L1 Data 128 KiB (x2)
  L1 Instruction 192 KiB (x2)
  L2 Unified 12288 KiB (x2)
Load Average: 0.25, 0.10, 0.02

```

Benchmark	Time	CPU	Iterations
normal	77.5 ns	77.5 ns	9040424
garbage	91.4 ns	91.4 ns	7675186

normal	73.1 ns	73.1 ns	9441284
garbage	90.6 ns	90.6 ns	7360351

Results on Windows with Microsoft (R) C/C++ Optimizing Compiler Version 19.40.33811 for ARM64 and Microsoft STL:

```
>double-benchmark.exe
2025-02-02T08:10:39-08:00
Running double-benchmark.exe
Run on (2 X 2000 MHz CPU s)
CPU Caches:
  L1 Instruction 192 KiB (x2)
  L1 Data 128 KiB (x2)
  L2 Unified 12288 KiB (x2)

-----
Benchmark      Time      CPU  Iterations
-----
normal         144 ns    143 ns    4480000
garbage        166 ns    165 ns    4072727
```

Although the output has the same size, producing "garbage digits" makes `std::format` 15-24% slower on these inputs. If we exclude string construction time, the difference will be even more profound. For example, profiling the benchmark on macOS shows that the `to_chars` call itself is more than 50% (!) slower:

```
garbage(benchmark::State&):
241.00 ms ... std::__1::to_chars_result std::__1::_Floating_to_chars[abi:ne180100]<...>(cha
normal(benchmark::State&):
159.00 ms ... std::__1::to_chars_result std::__1::_Floating_to_chars[abi:ne180100]<...>(cha
```

For comparison here are the results of running the same benchmark with `std::format` replaced with `fmt::format` which doesn't produce "garbage digits":

```
$ ./double-benchmark
Unable to determine clock rate from sysctl: hw.cpusfrequency: No such file or directory
This does not affect benchmark measurements, only the metadata output.
***WARNING*** Failed to set thread affinity. Estimated CPU frequency may be incorrect.
2025-03-15T08:18:56-07:00
Running ./double-benchmark
Run on (8 X 24 MHz CPU s)
CPU Caches:
  L1 Data 64 KiB
  L1 Instruction 128 KiB
  L2 Unified 4096 KiB (x8)
Load Average: 3.00, 3.91, 4.85

-----
Benchmark      Time      CPU  Iterations
-----
fmt_normal     53.0 ns    53.0 ns    13428484
fmt_garbage    53.4 ns    53.4 ns    13032712
```

As expected, the time is nearly identical between the two cases. It demonstrates that the performance gap can be eliminated if this paper is accepted.

Locale makes the situation even more confusing to users. Consider the following example:

```
std::locale::global(std::locale("en_US.UTF-8"));
auto s = std::format("{:L}", 1200000.0); // s == "1,200,000"
```

Here `s` is `"1,200,000"` even though `"1.2e+06"` would be shorter.

§ 3. Proposal

The current paper proposes fixing the default floating-point representation in `std::format` to use exponent range, fixing the issues described above.

Consistent, easy to reason about output format:

Code	Before	After
<code>std::format("{", 100000.0)</code>	<code>"1e+05"</code>	<code>"100000"</code>
<code>std::format("{", 120000.0)</code>	<code>"120000"</code>	<code>"120000"</code>

No "garbage digits":

Code	Before	After
<code>std::format("{", 1234567890123456700000.0)</code>	<code>"1234567890123456774144"</code>	<code>"1.2345678901234568e-</code>

Consistent localized output (assuming `en_US.UTF-8` locale):

Code	Before	After
<code>std::format("{:L}", 1000000.0)</code>	<code>"1e+06"</code>	<code>"1,000,000"</code>
<code>std::format("{:L}", 1200000.0)</code>	<code>"1,200,000"</code>	<code>"1,200,000"</code>

§ 4. Wording

Modify `[format.string.std]`:

Table 105 — Meaning of type options for floating-point types `[tab:format.type.float]`

Type	Meaning
<code>a</code>	If <i>precision</i> is specified, equivalent to <code>to_chars(first, last, value, chars_format::hex, precision)</code> where <code>precision</code> is the specified formatting precision; equivalent to <code>to_chars(first, last, value, chars_format::hex)</code>

	otherwise.
...	...
G	The same as g , except that it uses E to indicate exponent.
none	Let <code>fmt</code> be <code>chars_format::fixed</code> if <code>value</code> is in the range $[10^{-4}, 10^n)$, where 10^n is $2^{\text{std::numeric_limits}<\text{decltype}(\text{value})>::\text{digits} + 1}$ rounded down to the nearest power of 10, <code>chars_format::scientific</code> otherwise. If <code>precision</code> is specified, equivalent to <pre>to_chars(first, last, value, chars_format::general, precision)</pre> where <code>precision</code> is the specified formatting precision; equivalent to <pre>to_chars(first, last, value, fmt)</pre> otherwise.

§ 5. Implementation and usage experience

The current proposal is based on the existing implementation in [FMT] which has been available and widely used for over 6 years. Similar logic based on the value range rather than the output size is implemented in Python, Java, JavaScript, Rust and Swift.

§ 6. Impact on existing code

This is technically a breaking change for users who rely on the exact output that is being changed. However, the change doesn't affect ABI or round trip guarantees. Also reliance on the exact representation of floating-point numbers is usually discouraged so the impact of this change is likely moderate. In the past we had experience with changing the output format in [FMT], usage of which is currently at least an order of magnitude higher than that of `std::format`.

§ References

§ Informative References

[DRAGONBOX]

Junekey Jeon. *Dragonbox: A New Floating-Point Binary-to-Decimal Conversion Algorithm*. URL: https://github.com/jk-jeon/dragonbox/blob/master/other_files/Dragonbox.pdf

[FMT]

Victor Zverovich; et al. *The {fmt} library*. URL: <https://github.com/fmtlib/fmt>

[JAVA-DOUBLE]

Java™ Platform, Standard Edition 8 API Specification, Class Double, toString. URL: <https://docs.oracle.com/javase/8/docs/api/java/lang/Double.html#toString-double->

[N3220]

JeanHeyd Meneide; Freek Wiedijk; et al. *ISO/IEC 9899:2024. Information technology — Programming languages — C. 7.23.6 Formatted input/output functions*. URL: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3220.pdf#page=346>

[PYTHON-FORMAT]

The Python Standard Library, Format Specification Mini-Language. URL:
<https://docs.python.org/3/library/string.html#format-specification-mini-language>

[RYU]

Ulf Adams. *Ryū: fast float-to-string conversion*. 2018. URL: <https://dl.acm.org/doi/10.1145/3192366.3192369>

[STEELE-WHITE]

Guy L. Steele Jr.; Jon L White. *How to Print Floating-Point Numbers Accurately*. 1990.